

```
1 import org.junit.jupiter.api.BeforeEach;
2 import org.junit.jupiter.api.Test;
3
4 import java.awt.*;
5
6 import static org.junit.jupiter.api.Assertions.*;
7 public class TestCar {
8
9     //Objects to test
10    Car volvo;
11
12    @BeforeEach
13    void setUp() {
14        volvo = new Volvo240();
15    }
16
17    //Testing that the constructor creates a car with
correct settings
18    @Test
19    void testConstructor() {
20        assertEquals(4, volvo.getNrDoors());
21
22        assertEquals(100, volvo.getEnginePower());
23
24        assertEquals(Color.black, volvo.getColor());
25
26        assertEquals(0, volvo.getCurrentSpeed());
27
28        assertEquals(0, volvo.getX());
29
30        assertEquals(0, volvo.getY());
31
32        assertEquals("Volvo240", volvo.getModelName
33    ());
34    }
35
36    @Test
37    void TestEngine() {
38        volvo.startEngine();
39        assertEquals(0.1, volvo.getCurrentSpeed());
```

```
40
41     volvo.stopEngine();
42     assertEquals(0, volvo.getCurrentSpeed());
43 }
44
45 @Test
46 void testColor() {
47     volvo.setColor(Color.PINK);
48     assertEquals(Color.PINK, volvo.getColor());
49 }
50
51 @Test
52 void testGas() {
53     volvo.startEngine();
54     double initialSpeed = volvo.getCurrentSpeed
55 ();
56     volvo.gas(1);
57     assertTrue(initialSpeed < volvo.
58 getCurrentSpeed());
59
60     assertThrows(
61         IllegalArgumentException.class
62 , () -> volvo.gas(-1));
63
64     assertThrows(
65         IllegalArgumentException.class
66 , () -> volvo.gas(2));
67
68     double inSpeed = volvo.getCurrentSpeed();
69     volvo.gas(0.1);
70     assertFalse(inSpeed > volvo.getCurrentSpeed
71 ());
72 }
73
74 @Test
75 void testBrake() {
76     volvo.startEngine();
77     volvo.gas(1);
```

```
76
77     double initialSpeed = volvo.getCurrentSpeed
    ();
78     volvo.brake(1);
79
80     assertTrue(volvo.getCurrentSpeed() <
    initialSpeed);
81
82     assertThrows(
83         IllegalArgumentException.class,
84         () -> volvo.brake(-1));
85
86     assertThrows(
87         IllegalArgumentException.class,
88         () -> volvo.brake(2));
89
90     volvo.gas(0.1);
91     double intSpeed = volvo.getCurrentSpeed();
92
93     assertFalse(intSpeed < volvo.getCurrentSpeed
    ());
94 }
95
96
97 @Test
98 void testMove() {
99     volvo.startEngine();
100    volvo.gas(1);
101
102    double intX = volvo.getX();
103    double intY = volvo.getY();
104
105    volvo.move();
106
107    assertTrue(volvo.getY() > intY);
108    assertEquals(volvo.getX(), intX);
109 }
110
111 @Test
112 void turnLeft() {
113     volvo.startEngine();
```

```
114         volvo.gas(1);
115
116         double intX = volvo.getX();
117         double intY = volvo.getY();
118
119         volvo.turnLeft();
120         volvo.move();
121
122         assertTrue(volvo.getX() < intX);
123         assertEquals(volvo.getY(), intY);
124     }
125
126     @Test
127     void turnRight() {
128         volvo.startEngine();
129         volvo.gas(1);
130
131         double intX = volvo.getX();
132         double intY = volvo.getY();
133
134         volvo.turnRight();
135         volvo.move();
136
137         assertTrue(volvo.getX() > intX);
138         assertEquals(volvo.getY(), intY);
139
140     }
141
142     @Test
143     void testSaabTurboON() {
144         Saab95 saab = new Saab95();
145         saab.startEngine();
146         saab.gas(1);
147
148         double noTurboSpeed = saab.getCurrentSpeed
149     ();
150
151         saab.setTurboOn();
152         saab.gas(1);
153         assertTrue(noTurboSpeed < saab.
154             getCurrentSpeed());
```

```
153
154     }
155
156     @Test
157     void testSaabTurboOFF() {
158         Saab95 saab = new Saab95();
159         saab.startEngine();
160         saab.setTurboOn();
161         saab.gas(1);
162         double turboSpeed = saab.getCurrentSpeed();
163
164         saab.stopEngine();
165         saab.startEngine();
166         saab.setTurboOff();
167         saab.gas(1);
168         double noTurboSpeed = saab.getCurrentSpeed
169     ();
170         assertTrue(turboSpeed > noTurboSpeed);
171     }
172
173     @Test
174     void testVolvoTrimFactor() {
175         Volvo240 volvo = new Volvo240();
176         volvo.startEngine();
177
178         double speed1 = volvo.getCurrentSpeed();
179         volvo.gas(1);
180         double speed2 = volvo.getCurrentSpeed();
181
182         double actIncrease = speed2 - speed1;
183         double expIncrease = volvo.getEnginePower
184     () * 0.01 * 1.25;
185         assertEquals(expIncrease, actIncrease, 0.
186     0001);
187     }
188     @Test
189     void testRightCycle() {
190         volvo.startEngine();
```

```
191         double intY = volvo.getY();
192         double intX = volvo.getX();
193
194         volvo.turnRight();
195         volvo.turnRight();
196         volvo.turnRight();
197         volvo.turnRight();
198
199         volvo.gas(1);
200         volvo.move();
201
202         assertTrue(volvo.getY() > intY);
203         assertEquals(volvo.getX(), intX);
204     }
205
206     @Test
207     void testLeftCycle(){
208
209         volvo.startEngine();
210
211         double intY = volvo.getY();
212         double intX = volvo.getX();
213
214         volvo.turnLeft();
215         volvo.turnLeft();
216         volvo.turnLeft();
217         volvo.turnLeft();
218
219         volvo.gas(1);
220         volvo.move();
221
222         assertTrue(volvo.getY() > intY);
223         assertEquals(volvo.getX(), intX);
224     }
225
226     @Test
227     void testSanity() {
228         double intX = volvo.getX();
229         double intY = volvo.getY();
230
231         volvo.move();
```

```
232         assertEquals(intY, volvo.getY());
233         assertEquals(intX, volvo.getX());
234
235         double intSpeed = volvo.getCurrentSpeed();
236         volvo.gas(1);
237         assertEquals(intSpeed, volvo.getCurrentSpeed
    ());
238
239         volvo.startEngine();
240         volvo.gas(0.1);
241         volvo.stopEngine();
242
243         assertEquals(0, volvo.getCurrentSpeed());
244
245     }
246
247     @Test
248     void testSpeed() {
249         volvo.startEngine();
250         volvo.gas(1);
251         assertFalse(volvo.getCurrentSpeed() > volvo.
    getEnginePower());
252
253         volvo.brake(1);
254         assertFalse(volvo.getCurrentSpeed() < 0);
255
256     }
257
258 }
259
```